AR

PROJETO ACADÊMICO · CONTAINERS E ORQUESTRAÇÃO

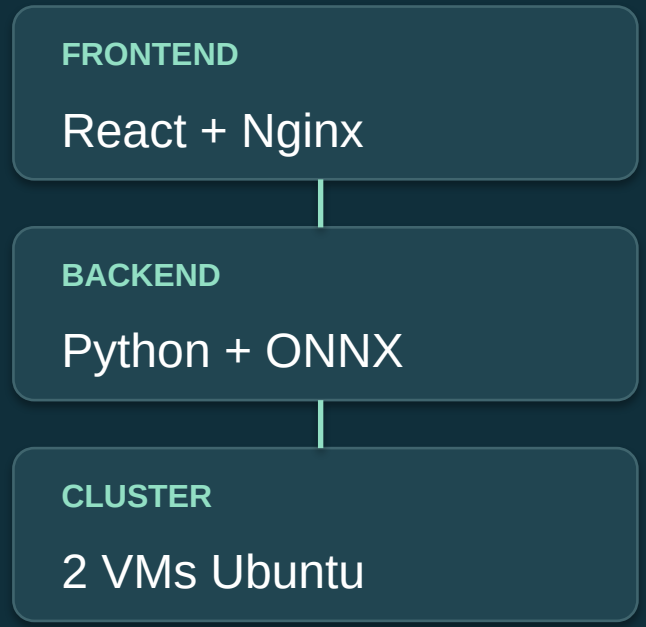
Auto Repair

Vistoria por fotos e orçamento de reparos, com análise local e revisão humana.

CPU LOCAL

DOCKER COMPOSE

SWARM + VISUALIZER



Da aplicação ao ambiente distribuído

01

Entender o sistema

Como fotos se tornam sugestões revisáveis e um orçamento organizado para o cliente.

02

Explicar a infraestrutura

Como as imagens, o Compose e o Swarm organizam a execução nas duas VMs.

Três requisitos do trabalho

1. Dockerfiles dos serviços utilizados.
2. Clusterização com Docker Swarm em VMs.
3. Compose, variação de réplicas e Docker Visualizer.

Organizar a vistoria antes de orçar

01 Fotos dispersas

Reunir visão geral e detalhes do veículo
na mesma vistoria, com foto de origem por item.

02 Identificação assistida

Usar o detector para sugerir regiões suspeitas.
A confirmação continua com a pessoa responsável.

03 Revisão dos valores

Separar a sugestão visual das cotações,
das horas de trabalho e da decisão de reparo.

04 Documento final

Consolidar itens revisados, mão de obra
e serviços de terceiros em um orçamento.

Objetivo: apoiar a organização do trabalho, sem substituir uma inspeção profissional.

Quatro etapas até a revisão do orçamento

01

Fotos

Visão geral e detalhes próximos.

→

02

Análise

Checar qualidade e rodar o detector.

→

03

Revisão

Corrigir, descartar ou incluir itens.

→

04

Orçamento

Ajustar preços, horas e serviços.

Depois da confirmação: salvar OS, compartilhar o orçamento ou exportar o PDF.

Uma boa foto é parte da análise

4
fotos por tentativa

8 MB
por arquivo

24 MP
por imagem

01 **Antes da inferência**

JPEG, PNG ou WebP. Correção da orientação e verificação de resolução, iluminação e foco.

02 **Preparação do modelo**

RGB, redução da imagem e letterbox para entrada de 640 × 640 pixels.

Fotos recusadas por qualidade não se transformam em resultados de danos.

O que o detector entrega — e seus limites

01 Motor de análise

ONNX Runtime em CPU, sem GPU ou PyTorch.
Pesos de 10,6 MB incluídos na imagem Docker.

02 Saída do modelo

Sugestões em 14 classes de peças,
com caixas na foto correspondente.

03 Modo rápido

Uma leitura da foto inteira.
Menor custo de processamento por imagem.

04 Modo detalhado

Foto inteira e até quatro recortes sobrepostos.
Mais busca de detalhes e maior custo de CPU.

Mais caixas não comprovam maior precisão. O modelo pode perder danos e gerar falsos positivos.

A foto não determina o valor do reparo

Conferência por item

Confirmar a região e o nome da peça.

Corrigir ou descartar erros do detector.

Definir o reparo e a forma de compra.

Revisar cotações e horas de trabalho.

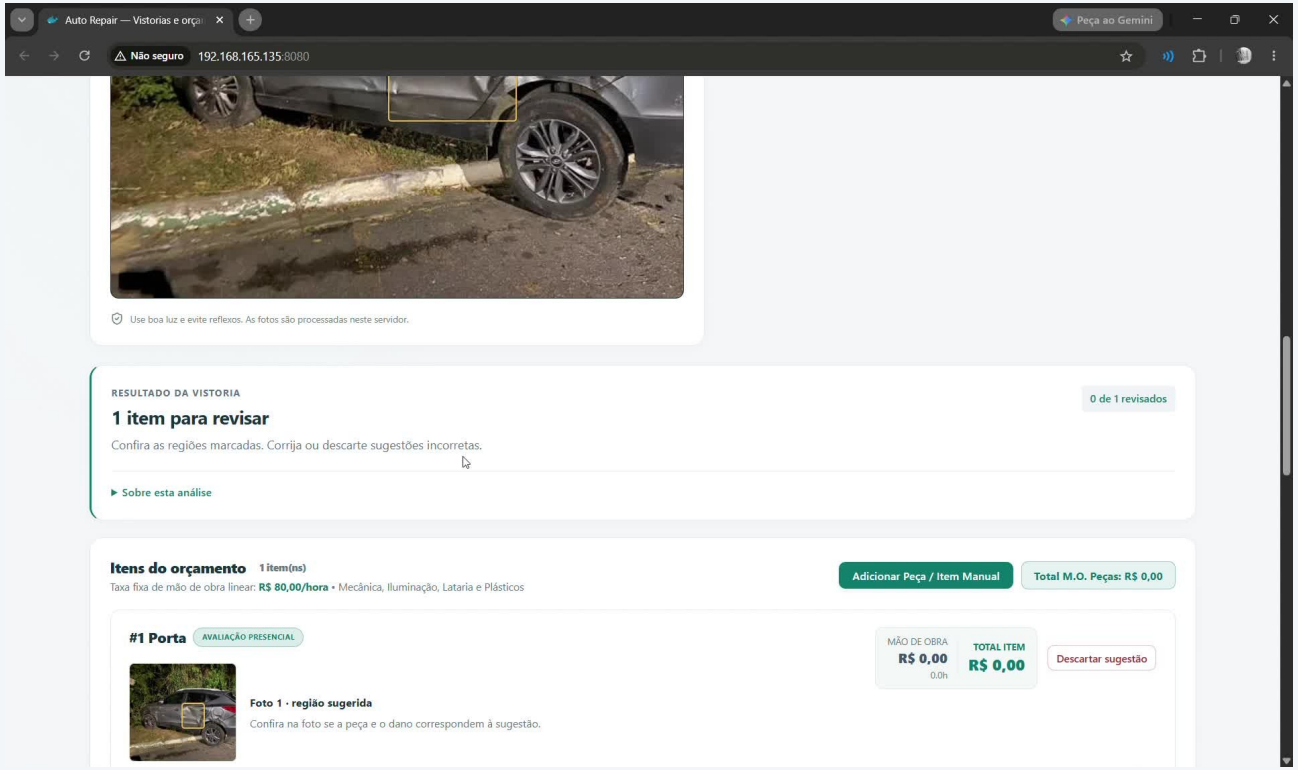
```
TOTAL DO ORÇAMENTO

Peças selecionadas
+ horas × R$ 80
+ serviços de terceiros
```

As referências iniciais não garantem estoque, compatibilidade ou procedência. Confirme as cotações antes de finalizar.

Da foto à primeira sugestão revisável

Captura real do vídeo em 00:35; aplicação acessada pelo manager em 192.168.165.135:8080.



O que foi observado

00:13 · Foto carregada.

00:29 · Análise em execução.

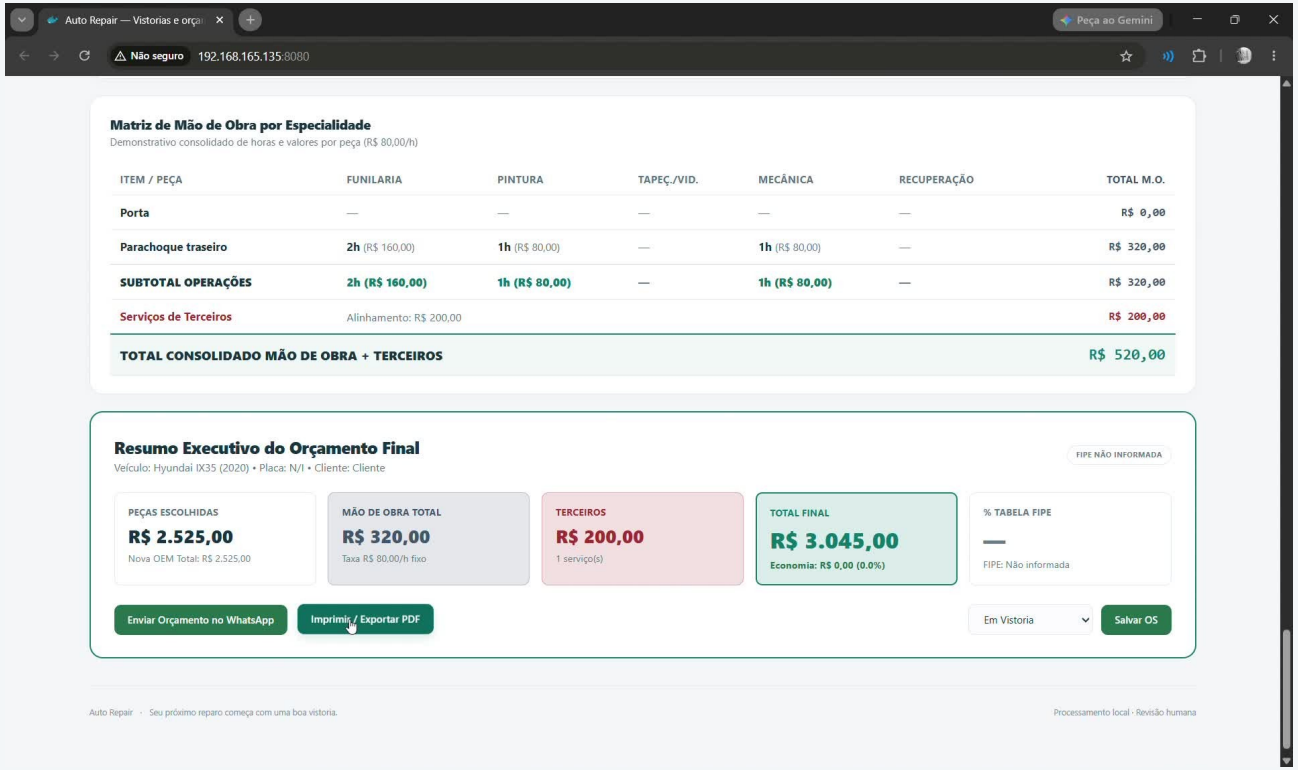
00:31 · Um item para revisão.

00:33–00:37 · Região na foto.

A sugestão observada foi Porta. Um único teste não mede a precisão do detector.

Revisão humana e orçamento consolidado

Captura real em 02:07. Os valores são dados do teste, não cotações verificadas.



Resultado exibido

Peças: R$ 2.525,00

Mão de obra: R$ 320,00

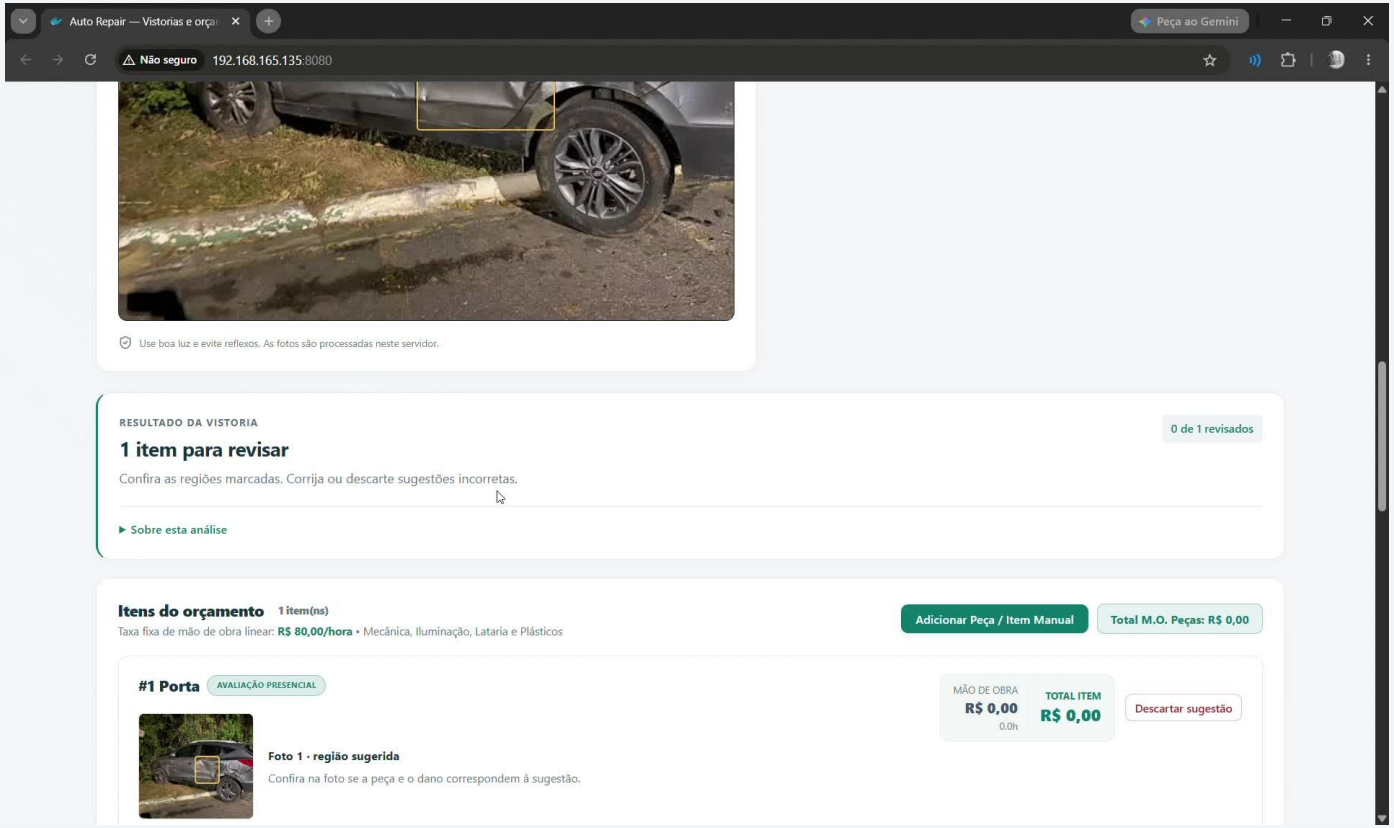
Terceiros: R$ 200,00

Total: R$ 3.045,00

O vídeo mostra inclusão manual de peça, ajustes e prévia de impressão; não mostra PDF salvo.

Primeiro teste completo da aplicação

Vídeo incorporado ao PowerPoint · 2min22s · sem narração · clicar para reproduzir.



No PDF, esta página é estática. Teste a reprodução no PowerPoint antes de apresentar.

Cada serviço tem uma responsabilidade

01 Frontend

React compilado em múltiplas etapas.

Nginx sem root, servindo a aplicação na porta 8080.

02 Backend

Python, ONNX Runtime e pesos verificados.

Análise local em CPU, sem API externa de IA.

03 Proxy de monitoramento

Consultas específicas à API Docker.

A porta 2375 não é publicada no host.

04 Docker Visualizer

Imagem dockersamples/visualizer, fixada por digest.

Execução no manager, com acesso pelo proxy.

```
bash scripts/build-images.sh --push
```

Multi-stage separa build e execução

FRONTEND

Node + Vite
Compilar a interface

→

Nginx
Servir os arquivos finais

BACKEND

Download + SHA-256
Preparar os pesos

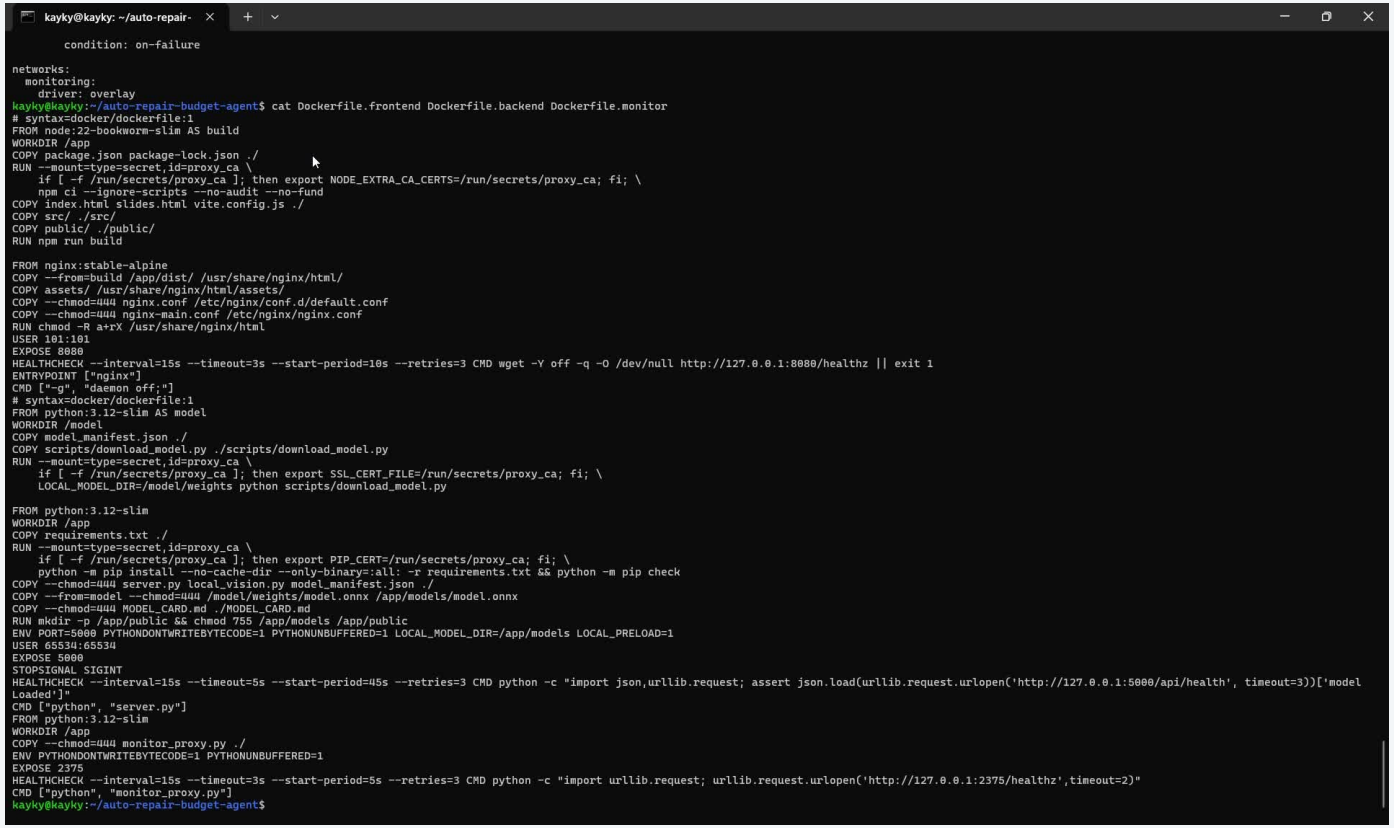
→

Python + bibliotecas
Executar a análise local

O runtime não precisa de Node para servir o front nem de internet para baixar o modelo.

Imagens e infraestrutura mostradas no terminal

Gravação incorporada · 49 segundos · clicar para reproduzir.



YAMLs da stack e os três Dockerfiles exibidos. A execução da escala pelo Compose é outra etapa.

Compose declara o estado desejado

```
deploy:
  replicas: ${APP_REPLICAS:-2}
  placement:
    max_replicas_per_node: 1
```

Healthchecks, limites de recursos, atualização gradual e rollback.

Arquivos da infraestrutura

docker-compose.yml

Frontend, backend e rede overlay.

docker-compose.demo.yml

Visualizer e proxy no manager.

compose.ubuntu.yml

Alternativa para um único Ubuntu.

Separação entre acesso e rede interna

01 Acesso à aplicação

8080: interface e proxy /api/.

8081: Visualizer, somente no manager.

Use a rede privada para a demonstração.

02 Serviços internos

5000: backend Python.

2375: proxy de monitoramento.

Essas portas não são publicadas no host.

COMUNICAÇÃO ENTRE AS VMs

2377/TCP · 7946/TCP e UDP · 4789/UDP

Duas VMs, uma rede de aplicação

MANAGER

192.168.165.135

- Frontend
- Backend
- Visualizer
- Proxy Docker

WORKER

192.168.165.140

- Frontend
- Backend

Executa tarefas distribuídas conforme o estado desejado.

Rede overlay · Aplicação: 8080 · Visualizer no manager: 8081

Um único manager não oferece alta disponibilidade do controle do cluster.

Manager coordena; worker executa

01 Manager

Gerencia serviços e o estado desejado.

Coordena o agendamento das tarefas.

Também pode executar frontend e backend.

Hospeda Visualizer e proxy.

02 Worker

Executa as tarefas atribuídas.

Participa da rede e do atendimento.

Não gerencia o plano de controle.

Recebe as mesmas imagens da aplicação.

Com uma réplica por serviço em cada nó, duas VMs comportam duas réplicas.

Se um nó cair, as tarefas podem ficar pendentes por causa dessa restrição.

Do código às imagens compartilhadas

01

Configurar o registro

Definir BACKEND_IMAGE, FRONTEND_IMAGE e MONITOR_IMAGE.

02

Construir e enviar

Gerar as imagens no manager e enviá-las ao GHCR ou registro escolhido.

03

Aplicar a stack

Executar o deploy no manager e aguardar convergência e saúde.

```
source .env.swarm
bash scripts/deploy-swarm.sh
```

O Visualizer consulta; o proxy filtra

VISUALIZER
Gráfico dos nós
e das tarefas

→

PROXY
Somente consultas
em rotas permitidas

→

DOCKER
Estado real
do manager

✓ Consultas permitidas

Nós, serviços e tarefas do cluster.
As respostas alimentam o gráfico do Visualizer.

× Alterações bloqueadas

POST, PUT, PATCH e DELETE são bloqueados.
O dashboard não recebe o socket real.

Socket montado com :ro, sozinho, não limita a API a leitura.
O painel de exemplo deve ficar na rede privada ou ser acessado por túnel SSH.

Roteiro declarativo para variar réplicas

INÍCIO

1

réplica(s) de cada serviço

→

EXPANSÃO

2

réplica(s) de cada serviço

→

REDUÇÃO

1

réplica(s) de cada serviço

```
bash scripts/demo-replicas.sh
```

Roteiro pelo Compose: 1 → 2 → 1. Nos prints recebidos, a redução 2 → 1 foi feita pela CLI.
A execução declarativa ainda precisa de um registro próprio.

Container em execução não basta

01 Prontidão do detector

Verificar e carregar o modelo antes
de abrir a porta do backend.

02 Healthchecks

Backend: API com modelLoaded true.
Frontend: resposta do Nginx em /healthz.

03 Atualização gradual

Uma réplica de cada serviço por vez.
As demais tarefas podem manter atendimento.

04 Rollback e diagnóstico

Falhas de atualização podem reverter a versão.
Consulte tarefas e logs antes de concluir.

Mostrar o cluster, não apenas os arquivos

```
docker node ls
docker stack services autobudget
docker stack ps --no-trunc autobudget
```

Coleta automatizada disponível

artifacts/swarm-demo/

Capturas do Visualizer: uma por etapa.

01 **Dois nós**

Manager e worker Ready/Active.

02 **Réplicas convergidas**

Prints: 2/2 → 1 pela CLI.
Compose: execução pendente.

03 **Execução da aplicação**

Foto analisada, revisão humana e orçamento.

Os identificadores técnicos autobudget_* foram preservados para compatibilidade.

Dos arquivos à execução nas duas VMs

Prints do chat e três vídeos revisados; resultados e limites de cada registro.

Etapa	Evidência	Situação
Cluster inicial	P01 + P02 + P03	Dois nós; serviços 2/2
Redução de réplicas	P04 + P05	Uma réplica por serviço
Queda do worker	P06 + P07	n2 Down; tarefas no manager
Aplicação / recuperação	V01 + V02 + V03	Análises vistas; retorno pendente

Prints originais do ZIP, capturas dos vídeos e sínteses identificadas nos slides seguintes.

Dois nós prontos no início

Print original P02 recebido em ZIP; identidades distintas de manager e worker.

```
root@kayky:/home/kayky/auto-repair-budget-agent# docker node ls
ID                            HOSTNAME   STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
qyent24b556eflmlcft6z2l7o *   kayky      Ready     Active         Leader           29.9.2
jl6nlu6l2520qua3o8wqxgv7g     n2         Ready     Active                          29.9.2
```

kayky · manager

Ready / Active / Leader

n2 · worker

Ready / Active

Docker Engine 29.9.2 nos dois nós; estado inicial do cluster.

Serviços convergidos antes do teste

Print original P03: docker stack services autobudget, antes da simulação de falha.

```
root@kayky:/home/kayky/auto-repair-budget-agent# docker stack services autobudget
ID             NAME                         MODE         REPLICAS               IMAGE
                    PORTS
9qxh35wkij4s   autobudget_backend           replicated   2/2 (max 1 per node)   ghcr.io/kaykymatos845/auto-repair-backend:v5
place08oyre1   autobudget_frontend          replicated   2/2 (max 1 per node)   ghcr.io/kaykymatos845/auto-repair-frontend:v5
                    *:8080->8080/tcp
tjqdk0xgap62   autobudget_monitor-proxy      replicated   1/1                    ghcr.io/kaykymatos845/auto-repair-monitor:v5
m55uexkjrha8   autobudget_visualizer        replicated   1/1                    dockersamples/visualizer@sha256:530c863672e7830d7560483df66beb4cbbcd375a9f3ec1
74ff5376616686a619
```

Aplicação · 2/2

Frontend e backend distribuídos.

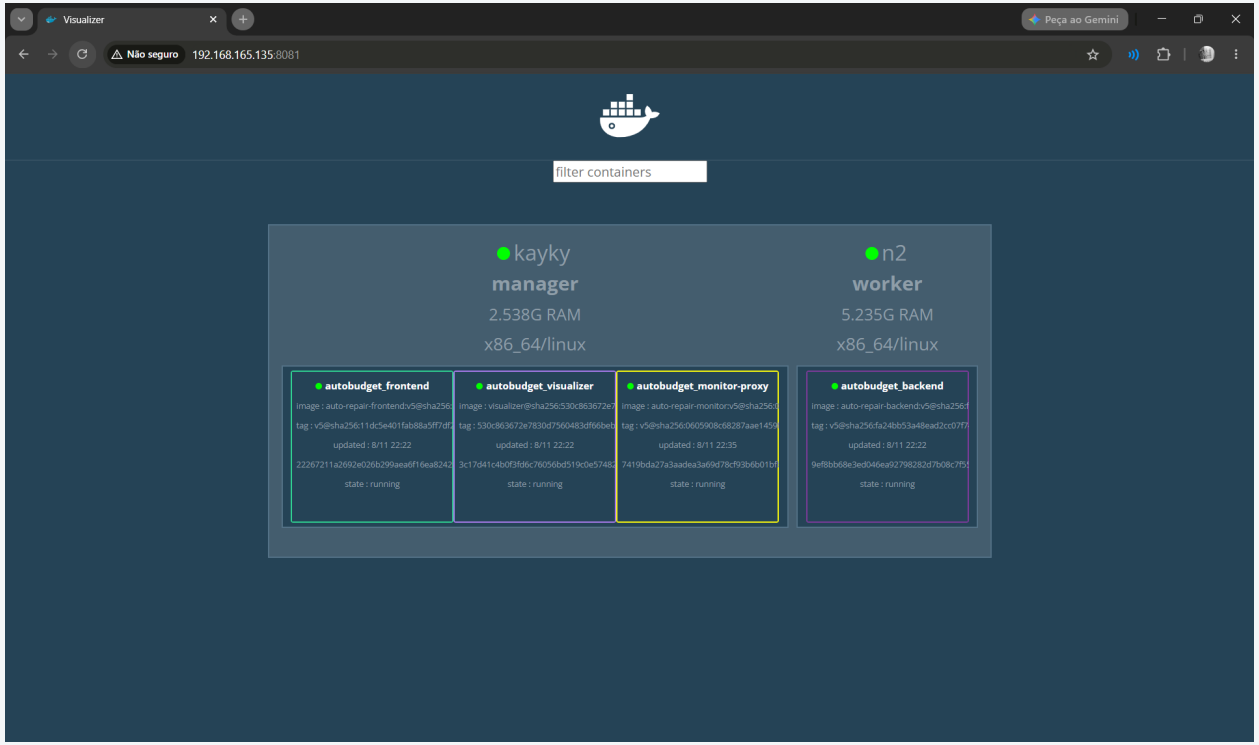
Monitoramento · 1/1

Visualizer e proxy no manager.

Imagens próprias v5 no GHCR; frontend publicado na porta 8080.

A distribuição muda com a escala

Print original P04 após a redução: frontend no manager e backend no worker.



Uma réplica por serviço

Manager: frontend e monitoramento.

Worker: backend.

Comunicação entre serviços pela rede overlay.

A redução está documentada pelo comando P05 e pelo resultado no Visualizer P04.

Redução executada pela linha de comando

O print registra uma alteração imperativa no estado dos serviços do Swarm.

```
root@kayky:/home/kayky/auto-repair-budget-agent# docker service scale autobudget_backend=1 autobudget_frontend=1
```

O que foi comprovado

Redução 2 → 1 e resultado visível no Visualizer.

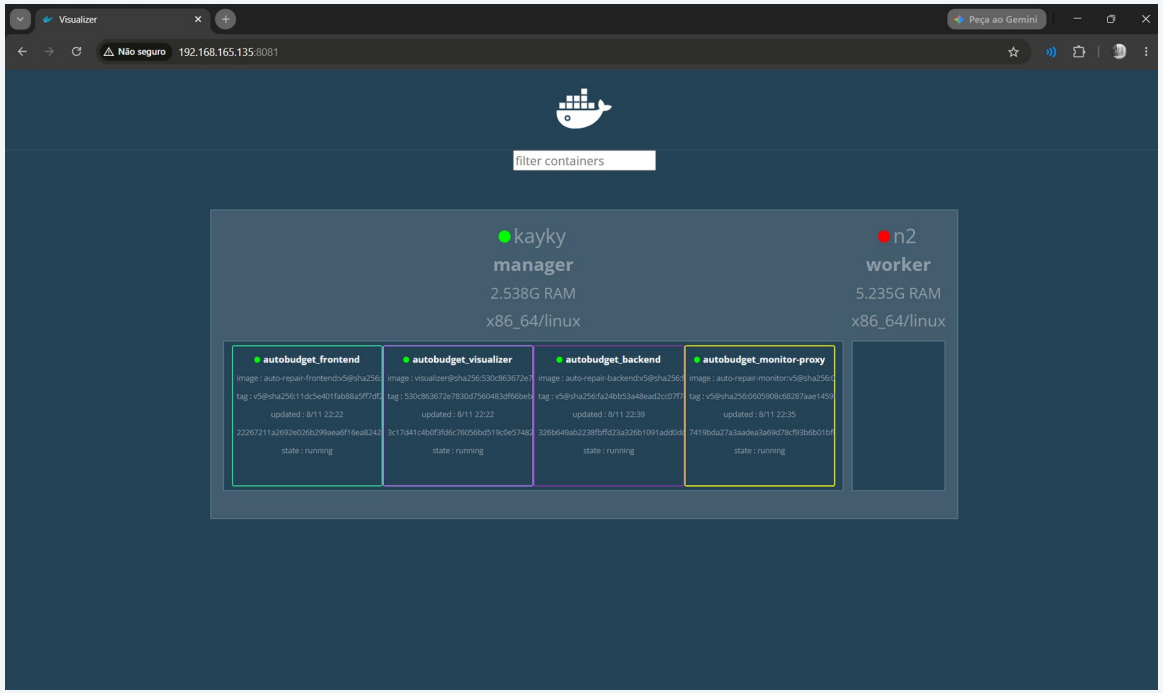
O que falta pelo Compose

Registrar APP_REPLICAS aplicado pelo script declarativo.

docker service scale não altera o YAML; o próximo deploy reaplica o valor do Compose.

Worker desligado; manager permanece ativo

Prints originais P06 e P07: Visualizer em vermelho e n2 Down no terminal.




```
root@kayky:/home/kayky/auto-repair-budget-agent# docker node ls
ID                            HOSTNAME   STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
qyent24b556eflmlcft6z2l7o *   kayky      Ready     Active         Leader           29.9.2
jl6nlu6l2520qua3o8wqxgv7g     n2         Down      Active                          29.9.2
```

Manager permanece Leader.

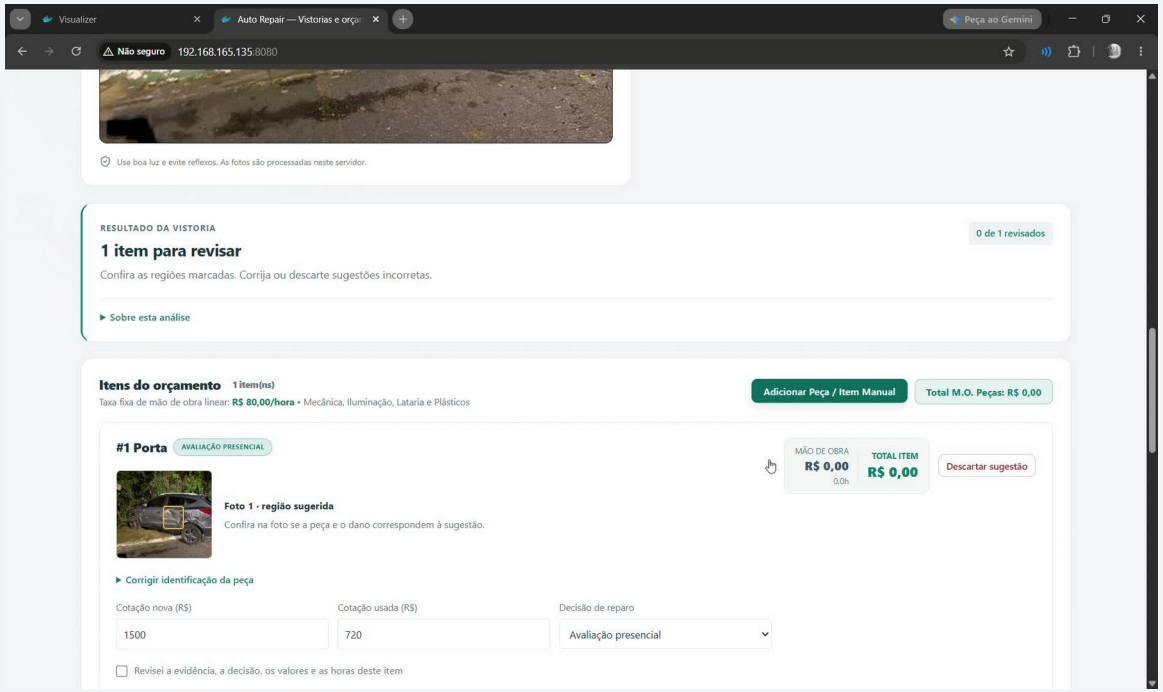
Worker aparece Down.

Frontend e backend aparecem no manager.

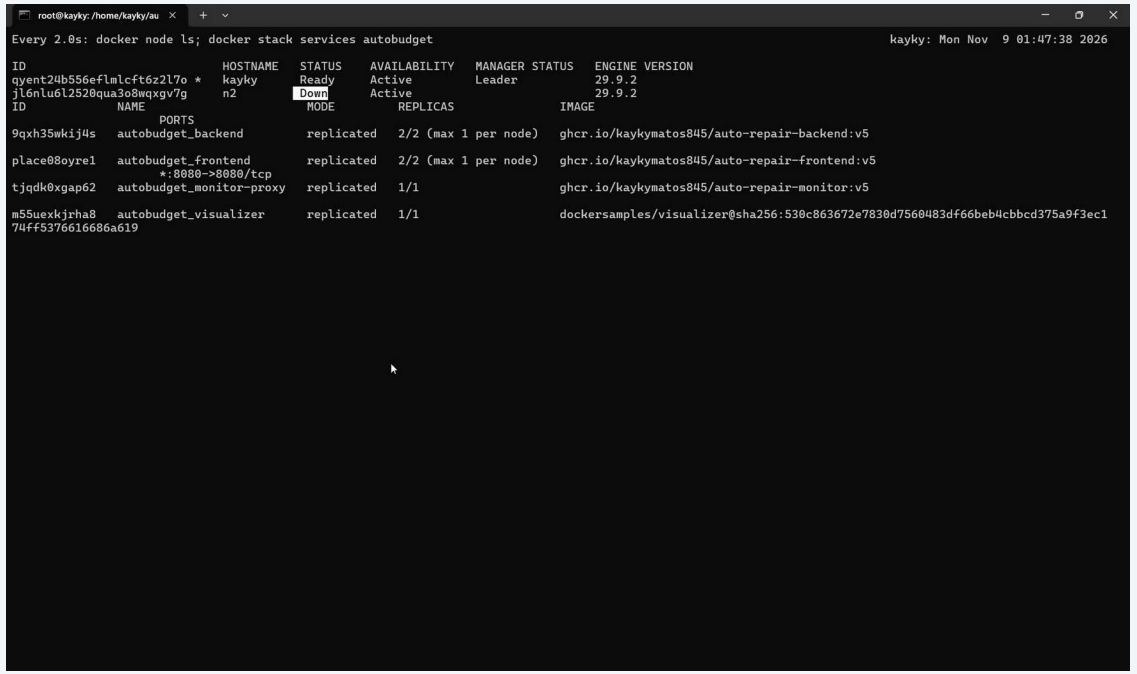
P06 mostra as tarefas remanescentes; V01 registra a análise relatada após a queda.

Nova análise e worker indisponível

Evidências complementares em gravações separadas; não há uma tela simultânea das duas.



V01 · 00:21 · Um item retornado pela análise.

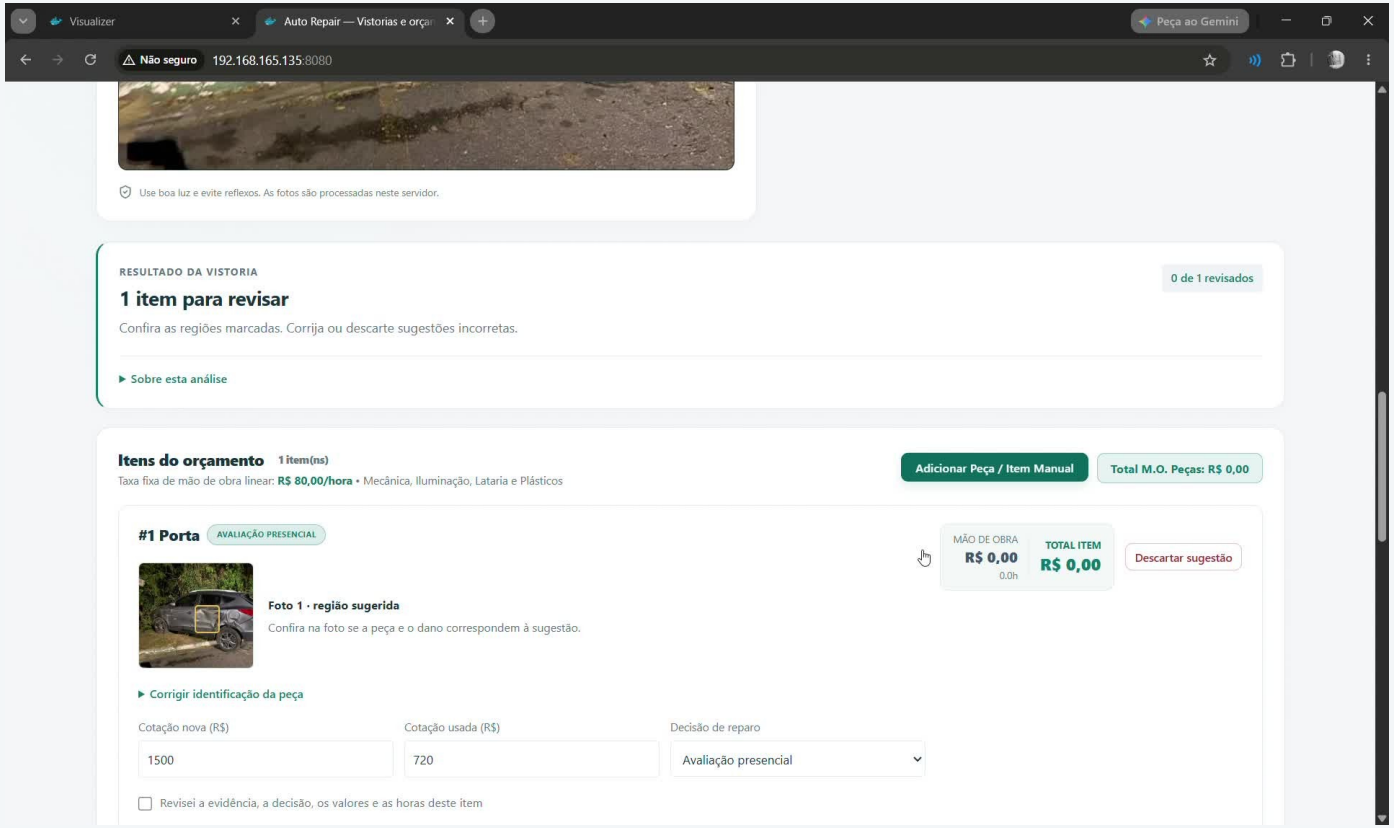


V02 · 00:25 · n2 Down; contagem exibe 2/2.

O autor identifica V01 como pós-queda; o vídeo do navegador não mostra o estado dos nós.

Aplicação no teste relatado após a queda

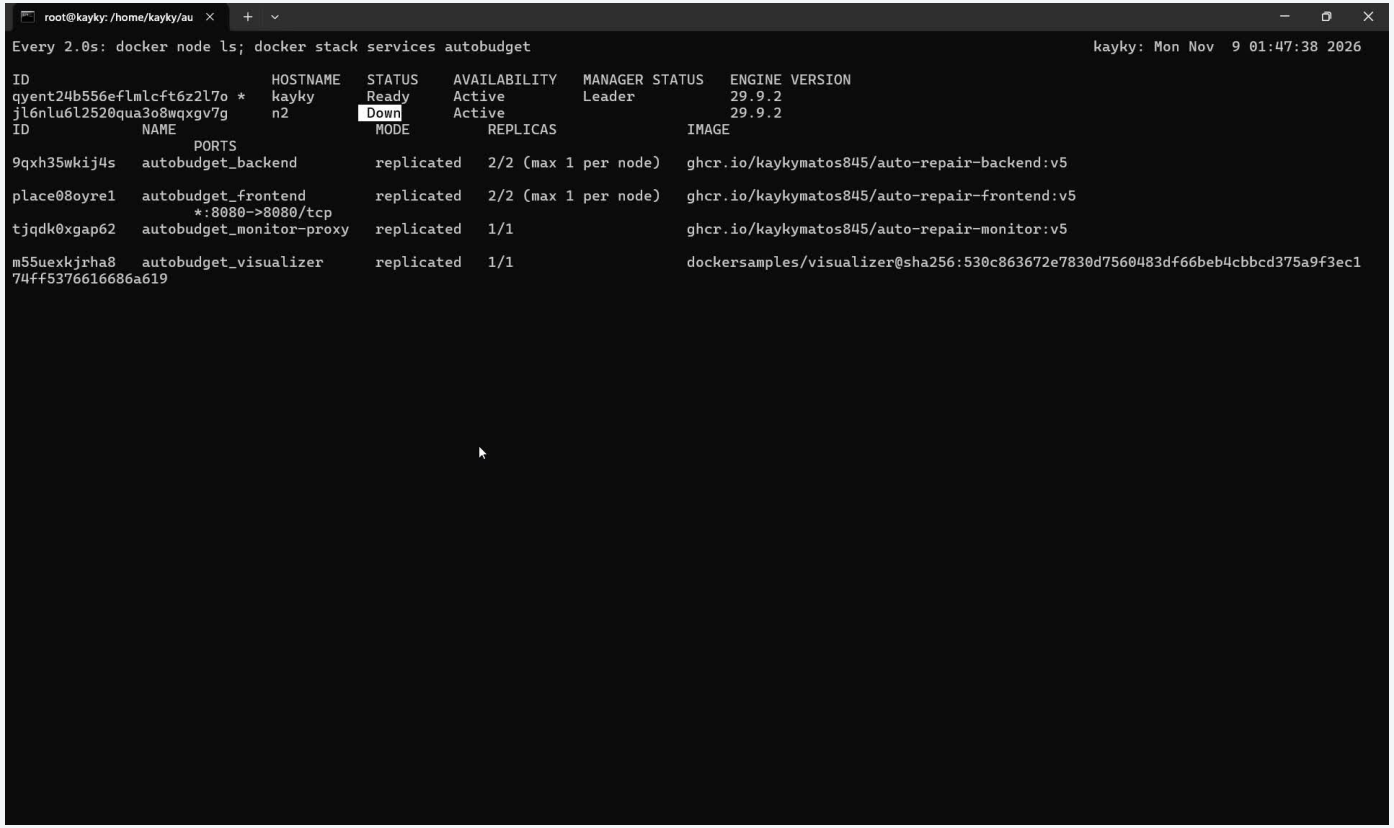
Gravação incorporada · 52 segundos · sem narração · clicar para reproduzir.



Mostra uma nova análise e orçamento. A associação à queda foi informada pelo autor.

Terminal durante a indisponibilidade do worker

Gravação incorporada · 58 segundos · sem narração · clicar para reproduzir.



n2 aparece Down. A contagem ainda mostra 2/2; verificar tarefas antes de afirmar convergência.

O que a queda permite concluir

O Visualizer comprova o estado dos nós e tarefas; a continuidade exige teste da aplicação.

Confirmado pelos prints

- Worker reconhecido como Down.
- Manager permanece Leader.
- Frontend e backend no manager.
- Monitoramento continua ativo.

Ainda precisa de evidência

- Estado das tarefas após a queda.
- Análise e nó Down simultâneos.
- Momento da recriação das tarefas.
- Worker recuperado e serviços 2/2.

Não afirmar migração, zero interrupções ou redundância do controle com um único manager.

Quatro gravações incorporadas à apresentação

V03: teste inicial. V01: teste após queda, segundo o autor. V02: terminal com n2 Down.

Arquivo	Nome da gravação	Data/hora no nome
V01	Chrome · Auto Repair	2026-11-08 22-42-24
V02	Terminal · root@kayky	2026-11-08 22-47-18
V03	Chrome · Auto Repair	2026-11-08 22-31-35
V04	Terminal · Dockerfiles e YAML	2026-11-09 20-03-01

V01: 52s · V02: 58s · V03: 2min22s · V04: 49s

As quatro gravações estão no arquivo; não dependem de links externos.

Mostre o efeito antes de explicar o comando

Sequência sugerida; os tempos são estimativas para a fala, não durações dos vídeos.

00:00–00:30	**Aplicação**	Foto → análise → revisão do resultado.
00:30–01:10	**Cluster**	Dois nós Ready; serviços 2/2; Visualizer.
01:10–02:00	**Escala**	Redução e expansão; comparar os gráficos.
02:00–03:00	**Falha**	n2 Down; nova análise pelo IP do manager.
03:00–03:40	**Recuperação**	Ligar n2; aguardar Ready e serviços 2/2.

Antes da queda: duas réplicas. Depois: análise nova. Ao final: worker recuperado.

O que falta para fechar a entrega

Itens objetivos para complementar a revisão e os registros já recebidos.

Prioridade	Registro necessário	Como obter
1	Recuperação do worker	n2 Ready + aplicação 2/2
2	Escala aplicada pelo Compose	scripts/demo-replicas.sh + capturas
3	Sincronismo do teste de falha	Análise + nó Down na mesma captura
4	Detalhar contagem durante falha	docker service ps --no-trunc

Vídeos revisados; falta sincronizar o teste de falha e mostrar a recuperação.

O que comprova cada parte do trabalho

Dockerfiles	Frontend, backend e proxy próprios; Visualizer usa a imagem de exemplo.
Swarm em VMs	Dois nós Ready/Active em docker node ls: manager e worker.
Compose e escala	Compose configurado; redução 2 → 1 registrada via CLI.
Representação gráfica	Prints: duas réplicas, uma réplica e worker indisponível.

As próximas evidências distinguem resultados observados e registros ainda pendentes.

ENCERRAMENTO

Infraestrutura verificável. Revisão humana.

01 Aplicação

Sugestões de regiões suspeitas.
Peça, preço, horas e decisão de reparo precisam de revisão.

02 Cluster

Um manager + um worker.
Histórico no navegador, sem banco compartilhado ou login na aplicação.

ENDEREÇOS DA DEMONSTRAÇÃO

Aplicação · 192.168.165.135:8080

Visualizer · 192.168.165.135:8081

Demonstração: cluster, escala e falha do worker registrados em prints.
Quatro vídeos revisados e incorporados. Falta registrar a recuperação.